Part 1: Executive Summary

In investment banking and capital markets, execution is only the first step of the trade lifecycle. Trade reconciliation is the critical middle-office control process where the bank's internal Front-Office trading records are cross-referenced against external statements provided by prime brokers or clearinghouses (the "Street" data). The objective is to ensure that what the firm believes it traded perfectly matches what actually cleared.

In high-volume trading environments, discrepancies—known as "breaks"—happen daily due to price slippage, partial fills, or dropped tickets. If these breaks are not identified and resolved before the T+1 or T+2 settlement deadline, the firm faces unknown market exposure, direct financial losses from unhedged positions, and heavy regulatory penalties. To mitigate these risks, I engineered a robust, automated reconciliation engine that ingests distinct datasets, runs algorithmic matching protocols, isolates exceptions based on their operational root causes, and instantly quantifies the exact capital at risk in INR so the risk management team can prioritize high-exposure breaks immediately.

Part 2: Technical Architecture & Data Flow

To replicate the rigid, controlled structure of an institutional back-office system, the project is divided into three dedicated Excel sheets, ensuring that raw data is never mixed with the final output:

- **The Internal_Ledger Sheet:** This acts as the firm's assumed baseline liability, representing trades executed and booked by internal front-office traders. It contains critical identifiers including the unique Trade ID, Security ISIN, Trade Date, expected Quantity, and the internally logged Execution Price.

- **The Broker_Statement Sheet:** Representing the external confirmation files (mimicking a SWIFT MT515 feed), this dataset details what actually cleared and settled on the street. Because external market data is notoriously messy, this sheet often contains formatting inconsistencies, partial fills, and unbooked trades that directly conflict with the internal ledger.

- **The Break_Report Sheet:** This centralized output matrix aligns the Internal and Broker data side-by-side to prevent analysts from having to stare across two screens. It categorizes the exact status of every trade (Matched Perfectly, Price/Quantity Break, Missing on Street, or the massive operational risk of being Missing Internal). For every break, it automatically computes the Net Exposure Diff (INR) by calculating the absolute variance between the internal expected value and the broker actual value, showing exactly how much capital is tied up in the discrepancy.

Part 3: Execution Strategy & Evolution

To ensure absolute accuracy and to deeply understand the financial logic before scaling it, this engine was developed in two distinct phases: a manual proof-of-concept followed by algorithmic VBA automation.

Phase 1 was built manually using advanced Excel functions. Because standard lookups cannot dynamically generate a list of missing rows, I first used dynamic arrays (=UNIQUE(VSTACK(...))) to stack both internal and broker IDs into a single master column. I then utilized VLOOKUP wrapped in IFERROR statements to pull respective metrics, and a complex, nested IF and COUNTIF logic gate to determine the exact break status. While mathematically accurate, this manual approach proved unscalable. Standard formulas are unidirectional and computationally heavy, and dragging thousands of array formulas bloated the file size while leaving the sheet vulnerable to accidental end-user deletion.

Phase 2 solved these speed and risk limitations by transitioning to the automated Run_Reconciliation_Engine VBA macro. Bypassing cell-level formulas entirely, the macro uses high-speed memory loops to execute a multi-pass algorithmic match. It runs a Forward Pass scanning internal trades against the street, followed by a critical Reverse Pass sweeping the broker statement backward to aggressively hunt down unbooked "Missing Internal" trades. To handle dirty external feeds, the script utilizes Trim() and Val() functions to forcefully convert text strings into pure mathematical values, eliminating system-crashing "Type Mismatch" errors. By optimizing the application memory, a manual reconciliation process that takes analysts hours is reduced to a single-click execution that runs in milliseconds.